

自定义协议的发布订阅式消息队列

通用问题

1. 为什么做这个项目？

在学习过程中, 接触到了 rabbitmq, 就很好奇 rabbitmq 是怎么实现的. 翻了下 rabbitmq 的源码, 发现是 erlang 语言实现的, erlang 的语法风格和 Java 差异很大(函数式编程, 使用模式匹配代替分支语句, 使用递归代替循环语句) 因此就想仿照它的核心功能, 尝试自己实现了下.

MQ 这个主题一般不会作为学校的课设. 同学们回答的时候建议还是从兴趣角度切入.

2. 这个项目为啥和上个(之前)同学的项目一样？

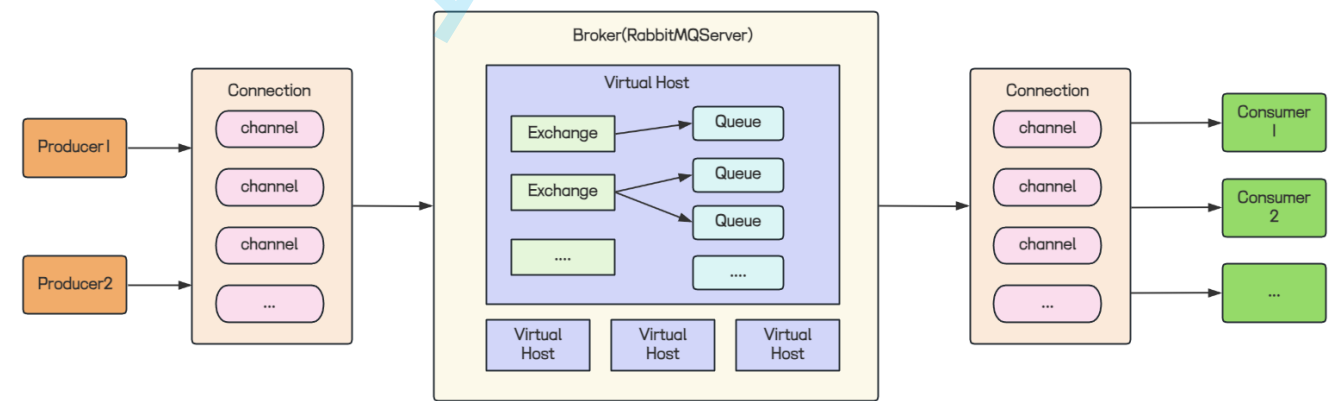
因为这个项目本身在实现的过程中, 在网上也搜索了一些相关的博客分析, 借鉴了一些开源项目, 写得优质的主要就是那几篇, 可能我们参考了一样的博客讲解梳理框架吧

(注: 以下内容需要同学自己思考项目扩展, 如不清楚, 不建议参照话术, 容易给自己挖坑. 说到上面的内容就可以了. 不要使用课上讲解的扩展, 大家都说一样的也不行)

但这个项目除了常规流程, 我还进行了(功能扩展/性能优化/代码优化)(并解释一下扩展点)

3. 介绍一下这个项目。包括一些核心概念及工作流程

参考: RabbitMQ的工作流程



RabbitMQ是一个消息中间件, 也是一个生产者消费者模型. 它负责接收, 存储并转发消息. 根据工作流程图, 来介绍它的核心概念:

- Producer: 生产者, 向RabbitMQ发送消息
- Consumer: 消费者, 从RabbitMQ接收消息

- Broker: 消息队列服务器或服务实例, 也就是RabbitMQ Server
- Connection: 网络连接, 它允许客户端与 RabbitMQ通信
- Channel: 连接里的一个虚拟通道, 发送或者接收消息都是通过通道进行的.
- Exchange: 交换机. 负责接收生产者发送的消息, 并根据路由算法和规则将消息路由到一个或多个队列
- Queue: 消息队列, 存储消息直到它们被消费者消费
- Binding: 交换机和队列之间的映射关系

工作流程:

1. 创建连接: Producer 连接到Broker, 建立一个连接(Connection), 开启一个信道(Channel)
2. 声明交换机和队列,以及绑定规则: Producer 声明一个交换机(Exchange)和队列, 并绑定Queue到Exchange
3. 发布消息: Producer 发送消息至 Broker
4. 消息存储: Broker 接收消息, 并存入相应的队列(Queue)中, 如果未找到相应的队列, 则根据生产者的配置, 选择丢弃或者退回给生产者.
5. 消费消息: 消费者监听Queue, 当消息到达时, 从Queue中获取消息, 处理后, 向RabbitMQ发送消息确认
6. 消息删除: 消息被确认后, Broker 会把消息从Queue中删除.

4. 项目最终的成果如何? 你从这个项目中学到了什么?

- 实现了消息队列的核心功能. 后续开发一些其他项目就可以考虑使用我自己的消息队列替换rabbitmq
- 最主要的收获是通过这个项目, 更深刻的理解了消息队列的使用方法和工作原理.
- 这个项目中基于 Socket 进行网络通信, 使我对于网络的底层原理有更清楚的认知, 尤其是自己设计应用层协议.
- 学到了当项目中遇到问题, 迫使我思考和寻找解决方案, 这增强了我的问题解决能力。
- 学到了如何进行有效的测试, 以确保项目不出现bug。

5. 你在这个项目中使用了哪些技术? 为什么选择(决策)这些技术?

- Spring: Java 后端主流技术
- Socket & 自定义应用层协议: 希望通过使用 Socket 加深对于网络底层操作/TCP协议的理解
- 数据结构/集合类: 组织数据必要组件

- 各种流对象(读写网络, 读写文件): 网络通信和持久化消息都需要进行 IO 操作
- SQLite 数据库: rabbitmq 在保存元数据时使用了 erlang 生态中的 Mnesia 数据库. 此处使用 mysql 存储太重量了, 后续部署会比较麻烦. 因此直接使用 sqlite 轻量的数据库更好的选择. (本身存储元数据, 数据量也不多, 数据结构也比较简单). 对于大量的数据(比如消息数据采取文件的方式来存储)

Mnesia 在英语中的发音是 mɪˈniːziə mɪˈniːziə。发音时, "m" 发音类似于 "m", "n" 发音类似于 "n", "i" 发音类似于 "ee" (如 "meet") , "z" 发音类似于 "z"。所以, 它听起来像 "mih-NEE-zee-uh"。

- unittest: spring 内置单元测试工具. 针对关键功能进行单元测试.

6. 你在项目中扮演了什么角色? 你负责了哪些模块或功能?

独立完成项目的设计, 开发, 测试, 部署

7. 在项目中, 你是如何平衡技术实现和项目时间线的? 这个项目做了多长时间?

这个项目我花了2个月来完成, 从项目目标的确定, 到项目原型图的敲定, 到前后端功能的拆分, 在到最后的开发测试部署。

关于时间, 其实我没有制定详细的规划。是因为我也是抱着学习的心态来完成这个项目。在开发中可能会遇到一些问题, 解决问题也是我学习的过程, 因此时间就不能确定下来。

回复思路可参考:

1. **明确项目范围和目标:** 在项目开始之前, 我确定了项目到底要做什么, 对范围、目标和期望有清晰的认知。
2. **制定详细的计划 (如果有):** 创建一个详细的项目计划, 包括项目核心功能敲定, 到模块划分, 前后端交互接口设计。
3. **优先级排序:** 确定哪些功能和特性是当下最重要的, 并根据优先级进行排序。这有助于在时间有限的情况下做出决策。
4. **人员分配 (如果有):** 由于前端是交付给其他同学完成, 以上内容也需要和前端同学对齐。(如果说前端也是自己完成的, 就不用说这一点) 本项目不涉及前端, 其他项目可以参考。
5. **沟通和协作 (如果有):** 确保和前端同学有良好的沟通和协作, 以便快速解决问题并共享信息。(如果说前端也是自己完成的, 就不用说这一点)
6. **质量保证:** 计划中还包含足够的时间来测试项目, 进行质量保证。

8. 你在项目中遇到了哪些挑战? 你是如何识别和解决这些问题的?

- 在调研阶段, 阅读 rabbitmq 的源码, 挑战较大. Rabbitmq 的代码是 erlang 语言开发, 语法风格和 java 差异较大 (使用模式匹配代替条件语句, 使用递归代替循环语句). 后来通过网上搜集资料, 了解 rabbitmq 的各种核心特性, 尝试自己对照功能来进行实现
- 开发阶段针对 客户端 服务器 之间的通信方式的实现, 由于决定采用 tcp socket, 就需要手动实现一些连接管理, 自定义协议处理粘包问题, 支持数据报文和控制报文等细节逻辑. 这里是花了较多的时间设计开发的.
- 测试阶段遇到了 xx bug, 经过一系列的定位过程, 最终找到了问题, 进行了修复.

上述只是提供一个切入的思路. 具体如何回答, 每个同学要结合自己做项目过程中的感受来组织语言. 面试官问这样的问题本质上就是试探同学是否是真的把项目做出来了, 还是抄来的.

切忌凭空编造! 很容易露馅!

9. 你是如何测试和部署项目的?

测试:

1. 单元测试: 编写测试代码来验证每个独立模块或函数的正确性。
2. 集成与功能测试: 在单元测试之后, 测试各个模块如何一起工作, 以及功能接口是否正确返回。
3. 日志打印: 使用日志打印来记录、分类和修复发现的问题。

部署:

1. 构建自动化: 通过 Maven 来编译代码和生成部署包。
2. 环境准备: 确保部署环境 (开发、生产) 都已正确设置。
3. 部署: 将包发布在阿里云服务器并后台启动
4. 日志: 部署后收集日志以便于问题诊断。

10. 你是如何优化项目性能的? 你有哪些性能调优的经验?

- 比如在网络通信这里, 使用二进制协议代替 json 这样的文本协议, 能够缩减传输数据量, 节省网络通信带宽.
- 比如在管理数据这里, 通过选择合适的数据结构 (比如 hashmap), 优化数据访问的效率
- 另外, 还可以对项目进行压测, 通过添加日志, 记录各个关键环节的时间戳, 找到性能瓶颈, 再做针对性的优化.

11. 你是如何设计项目以支持未来的扩展? 你是如何确保代码的可维护性的?

设计应用层协议阶段,就需要对协议的扩展性通用性进行考虑.

此处核心设计思路是 "操作码" + "操作参数" 这样的模式, 后续如果需要扩展更多的功能, 协议格式整体不需要变动, 只需要添加新的操作码, 并且针对性的添加操作参数即可.

专属问题

1. 你了解过其他一些消息队列吗? 你的消息队列相较于其他的有何区别或优势?

类似问题:

了解过哪些MQ, 与其他同类产品的对比

kafka 和 RabbitMQ的对比

对比其他消息队列, 不同mq分别用在什么场景

kafka和rocketmq比较

消息队列除了使用RabbitMQ, 可以用RocketMQ吗?

目前业界有很多的MQ产品, 例如RabbitMQ, RocketMQ, ActiveMQ, Kafka, ZeroMQ等,

简单介绍其中3种:

1. Kafaka

Kafka一开始的目的就是用于日志收集和传输, 追求高吞吐量, 性能卓越, 单机吞吐达到十万级, 在日志领域比较成熟, 功能较为简单, 主要支持简单的 MQ 功能. 适合大数据处理, 日志聚合, 实时分析等场景

2. RabbitMQ

采用Erlang语言开发, MQ 功能比较完备, 且几乎支持所有主流语言, 开源提供的界面也非常友好, 性能较好, 吞吐量能达到万级, 社区活跃度较高, 文档更新频繁, 比较适合中小型公司, 数据量没那么大, 且并发没那么高的场景.

3. RocketMQ

采用Java语言开发, 由阿里巴巴开源, 后捐赠给了Apache. 在可用性, 可靠性以及稳定性等方面都非常出色, 吞吐量能达到十万级, 在Alibaba集团内部广泛使用, 但支持的客户端语言不多, 产品较新文档较少, 且社区活跃度一般. 适合于大规模分布式系统, 可靠性要求高, 且并发大的场景, 比如互联网金融.

这些消息队列, 各有侧重, 没有好坏, 只有适合不适合, 在实际选型时, 需要结合自身需求以及MQ产品特征, 综合考虑

我自己的消息队列只是为了满足自己的好奇心, 从优势上来讲肯定是不如人家成熟的商业产品的.

2. 这个项目借鉴了哪个消息队列？为什么选择这个？

借鉴了 RabbitMQ

RocketMQ 的核心概念设定和 RabbitMQ 大同小异.

Kafka 部署起来比较麻烦 (依赖zookeeper, 并且需要多节点部署), 对我们低配置的阿里云学生机不友好, 学习起来不像 RabbitMQ 这么方便

因此在学习阶段主要学习的 RabbitMQ, 所以在做这个项目的时候也就是主要参考 RabbitMQ 的设定.

3. MQ 的作用及应用场景

类似问题:

项目什么场景下使用到了MQ, 为什么需要MQ

MQ 的作用? 使用场景有哪些

MQ的主要应用场景

消息队列解耦应用程序的例子

消息队列的应用场景

为什么说消息队列可以削峰

消息队列 (MQ) 是一种应用程序间的通信方法, 它允许系统组件以异步的方式进行交互, 在不同的应用场景下可以展现不同的作用, 常见的应用场景如下:

- **异步解耦:** 在业务流程中, 一些操作可能非常耗时, 但并不需要即时返回结果. 可以借助MQ把这些操作异步化, 比如 用户注册后发送注册短信或邮件通知, 可以作为异步任务处理, 而不必等待这些操作完成后才告知用户注册成功.
- **流量削峰:** 在访问量剧增的情况下, 应用仍然需要继续发挥作用, 但是是这样的突发流量并不常见. 如果以能处理这类峰值为标准而投入资源,无疑是巨大的浪费. 使用MQ能够使关键组件支撑突发访问压力, 不会因为突发流量而崩溃. 比如秒杀或者促销活动, 可以使用MQ来控制流量, 将请求排队, 然后系统根据自己的处理能力逐步处理这些请求.
- **异步通信:** 在很多时候应用不需要立即处理消息, MQ提供了异步处理机制, 允许应用把一些消息放入MQ中, 但并不立即处理它,在需要的时候再慢慢处理.
- **消息分发:** 当多个系统需要对同一数据做出响应时, 可以使用MQ进行消息分发. 比如支付成功后, 支付系统可以向MQ发送消息, 其他系统订阅该消息, 而无需轮询数据库.
- **延迟通知:** 在需要在特定时间后发送通知的场景中, 可以使用MQ的延迟消息功能, 比如在电子商务平台中, 如果用户下单后一定时间内未支付, 可以使用延迟队列在超时后自动取消订单
-

4. 项目中如何保证消息的可靠性？如果没有实现，说一说思路。

Consumer 成功消费, 会给 broker 发送一个应答报文.

应答的模式实现了手动应答和自动应答. 可以根据需要选择.

- 如果对于可靠性要求不高, 但是对吞吐性能要求较高, 就使用自动应答.
- 如果对于可靠性要求较高, 但是对吞吐性能要求不高, 就使用手动应答.

5. 项目中如何保证消息的顺序性？如果没有实现，说一说思路。

在项目中没有实现顺序性.

如果要实现, 需要给消息按照顺序编号, producer 每次发送的数据, 编号都需要累加, consumer 收到的数据按照编号在接收方进行排序

参考 TCP 协议的实现.

6. 项目中如何保证消息消费时的幂等性？如果没有实现，说一说思路。

幂等性一般是业务逻辑保证 (某个业务, 比如计费, 是幂等的). 对于 MQ 本身来说一般不提供幂等支持.

如果要实现的话, 需要给业务请求分配唯一 ID (不同于刚才网络传输的消息序号, 不要求递增, 往往可能是一个 UUID 类似的东西), 并且在消费者这一端通过一些方式 (数据库/redis/内存hash) 记录已经处理过的请求 ID. 避免重复处理.

7. 当消息过期或被拒绝，又或者队列满了，如何处理？如果没有实现，说一说思路。

类似问题:

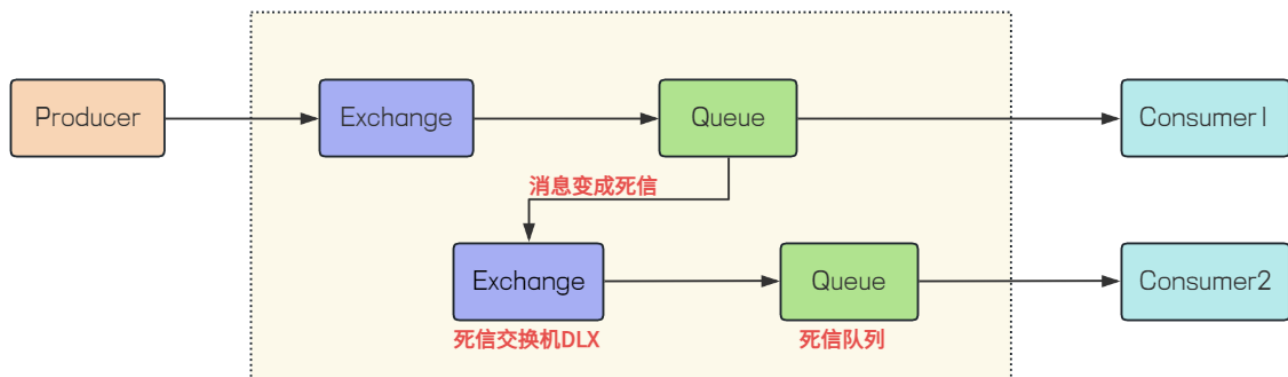
你了解 RabbitMQ 吗？介绍下 RabbitMQ 的死信队列

从以下方面来回答:

- 死信队列的概念
- 死信的来源
- 死信队列的应用场景

1. 死信队列的概念

死信 (Dead Letter) 是消息队列中的一种特殊消息, 它指的是那些无法被正常消费或处理的消息. 在消息队列系统中, 如RabbitMQ, 死信队列用于存储这些死信消息



2. 死信的来源

- 1) 消息过期: 消息在队列中存活的时间超过了设定的TTL
- 2) 消息被拒绝: 消费者在处理消息时, 可能因为消息内容错误, 处理逻辑异常等原因拒绝处理该消息. 如果拒绝时指定不重新入队(requeue=false), 消息也会成为死信.
- 3) 队列满了: 当队列达到最大长度, 无法再容纳新的消息时, 新来的消息会被处理为死信.

3. 死信队列的应用场景

对于RabbitMQ来说, 死信队列是一个非常有用的特性. 它可以处理异常情况下, 消息不能够被消费者正确消费而被置入死信队列中的情况, 应用程序可以通过消费这个死信队列中的内容来分析当时所遇到的异常情况, 进而可以改善和优化系统.

比如: 用户支付订单之后, 支付系统会给订单系统返回当前订单的支付状态

为了保证支付信息不丢失, 需要使用到死信队列机制. 当消息消费异常时, 将消息投入到死信队列中, 由订单系统的其他消费者来监听这个队列, 并对数据进行处理(比如发送工单等, 进行人工确认).

场景的应用场景还有:

- 消息重试: 将死信消息重新发送到原队列或另一个队列进行重试处理.
- 消息丢弃: 直接丢弃这些无法处理的消息, 以避免它们占用系统资源.
- 日志收集: 将死信消息作为日志收集起来, 用于后续分析和问题定位.

8. 消息积压如何处理? 什么原因可能会导致消息积压?

类似问题:

MQ消息堆积问题

如何解决MQ的数据囤积?

消息积压的原因:

- 消息生产过快
- 消费者能力不足

- 网络问题
- RabbitMQ服务配置偏低
-

解决方案:

- 提高消费者效率, 比如增加机器, 优化业务逻辑
- 限制消费者生产速率
- 资源配置优化

9. 消息持久化后, 若磁盘已满如何处理? 有哪些实现策略或思路?

思路参考:

1. **清理磁盘空间:** 检查并删除不必要的文件, 或者移动文件到其他存储设备上, 以释放磁盘空间。如果是日志文件占用了大量空间, 可以考虑压缩或删除旧的日志文件。
2. **重新分配队列数据文件:** 如果MQ因为磁盘满而无法启动, 可能需要手动删除队列数据文件中可能损坏的文件。删除前请确保备份重要数据, 因为这一操作会导致队列中的数据丢失。
3. **增加磁盘容量:** 如果经常遇到磁盘空间不足的问题, 可能需要考虑增加物理磁盘或调整虚拟机的磁盘配额。
4. **监控和报警:** 设置监控和报警机制, 以便在磁盘空间不足时及时收到通知并采取措施。
5. **优化消息处理:** 优化消息的生产和消费逻辑, 减少消息积压, 从而减少磁盘上消息的存储。
6. **使用不同的存储路径:** 如果服务器上有多个磁盘或分区, 可以考虑将MQ的数据目录迁移到有更多可用空间的磁盘或分区上。

10. 项目中消息传递是推模式还是拉模式? 如何实现?

推模式.

消费者和broker 之间建立了 tcp 长连接. Broker 收到数据之后, 就会依据订阅管理模块, 找到队列对应的消费者客户端的 socket 对象, 并且把队列中的数据读出来, 写入socket, 从而传输给对应的 consumer.

推模式实时性更好, 但是在数据量大的时候可能对 consumer 造成较大的压力.

Rabbitmq 默认的消费模式就是推模式.

如果要实现拉模式, 只需要让 broker 提供 tcp 接口, 让 consumer 主动给 broker 发起请求获取数据即可.